SimQC – Weekly Progress Report

Aoutul Nabi Purna*, Shishir Dhar†, Shihab Sharar‡, and Md. Iftikhar Alam Hridik§
Department of Electrical and Computer Engineering, North South University, Dhaka, Bangladesh
Emails: {*aoutul.purna.241, †shishir.dhar.241, ‡shihab.sharar.241, §iftikhar.hridik.241}@northsouth.edu

***Abstract*—This report outlines Sprint 3 of the SimQC quantum simulator, covering the engine analysis core, the tiered step trace, qubit-ceiling limits, QFT and Shor algorithm circuit builders, circuit extensions (URL codec, subcircuit nesting and recursive flattening, undo/redo history), and two read-only leaf modules—an inspector (purity, Bloch vector, shot histogram) and a debugger (timeline scrubber, breakpoint manager). The sprint hardens the core pipeline of the zero-allocation state-vector engine: every source module carries a 1:1 mirrored test, `node --test` passes 100%, and the headless CLI emits correct Bell-state JSON on `stdout`. The `ai/` module and DOM-bound `canvas/` panels are deferred to Sprint 4.**

I. Progress Summary

During this sprint we implemented build-order steps 5, 7–10 per the frozen Sprint 3 plan[cite: 1]. Completed deliverables include:

1) `src/sim/analytic.js` — curated closed-form state expectations (Bell, GHZ, QFT phase transforms, teleportation). `verify(analyticExpectation)` compares trace states to formulas within 10^{-12} for $n \leq 8$ and 10^{-9} for $n > 8$.
2) `src/sim/StepTrace.js` — tiered step trace over circuit columns. Dense Mode ($n \leq 12$) clones state pairs at column boundaries for $O(1)$ scrubbing; Ephemeral Replay Mode ($n > 12$) uses a single buffer pair with fast-forward on `seek()`. API: `reset()`, `next()`, `seek(targetCol)`, `findStepWhere(predicate)`, `verify()`, and read-only `currentState()` accessor.
3) `src/sim/limits.js` — qubit-ceiling thresholds `MAX_DENSE_QUBITS=12`, `MAX_EPHEMERAL_QUBITS=20`, `HARD_CLI_CAP=24`; `evaluate()` emits an advisory warning for $15 \leq n \leq 20$.
4) QFT (`src/algorithms/qft.js`) and Shor (`src/algorithms/shor.js`) circuit builders, verified against `analytic.js` phase expectations; Step 5 verification confirms ≥ 3 canonical algorithms (Deutsch–Jozsa from Sprint 2, plus QFT and Shor) pass closed-form distribution checks.
5) `src/circuit/Serializer.js` — Tier 2 URL codec round-trip: `deflate-raw` plus URL-safe Base64 (`[A-Za-z0-9-_]`, no `=` padding), lossless across circuits with subcircuits, controls, and anti-controls.
6) `src/circuit/GateColumn.js` — Gate carries an optional embedded `Circuit` (`subcircuit`/`subcircuitName`); `isEqualTo` compares subcircuits structurally.
7) `src/circuit/Flatten.js` — `flattenCircuit` recursively inlines nested subcircuits in deterministic left-to-right, top-to-bottom order; `virtualColumn` metadata shifts correctly.
8) `src/state/History.js` — linear stack of immutable `Circuit` references; `undo()`/`redo()` round-trip over 100 random operations via `isEqualTo()`.
9) `src/inspector/` (read-only leaf) — `Inspector.js` amplitude/probability readouts from `StepTrace.currentState()`; `Purity.js` single-qubit purity $\gamma_q = \mathrm{Tr}(\rho_q^2)$ and linear entropy $S_L(q) = 1 - \gamma_q$ writing to pre-allocated `Float64Array` buffers; `Bloch.js` per-qubit vector $\vec{r}_q = (2\Re[\rho_{01}], -2\Im[\rho_{01}], \rho_{00} - \rho_{11})$; `Histogram.js` shot sampler returning count maps summing to shots. Zero mutations; no imports from `debugger/` or `ai/`.
10) `src/debugger/` (read-only leaf) — `Debugger.js` playback controls (`reset`/`next`/`seek`); `BreakpointManager.js` `findStepWhere` breakpoint toggle; `Timeline.js` scrubber with $O(1)$ snapshot copy (Dense Mode) or zero-allocation replay (Ephemeral Mode). Zero mutations; no imports from `inspector/` or `ai/`.

The `ai/` folder (step 11) was removed and is not touched this sprint; DOM-bound `canvas/` panels (step 6) are deferred to Sprint 4. The frozen conventions `CONVENTIONS.md`, `FOLDER_STRUCTURE.md`, and `SOURCE_OF_TRUTH.md` remain the source of truth[cite: 1].

II. Learning: Tiered Step Trace and Read-Only Leaves

The textbook approach of materializing every simulation snapshot at each column costs $\mathcal{O}(n \cdot 2^n)$ memory and prohibits interactive scrubbing at scale[cite: 1]. Sprint 3's `StepTrace` avoids this with a tiered strategy: for $n \leq 12$ it retains cloned state pairs at column boundaries (Dense Mode, $O(1)$ random access); beyond that it keeps one buffer pair and fast-forwards on demand (Ephemeral Replay Mode, zero-allocation) [**?**]. This mirrors the matrix-free pairwise kernel from Sprint 2, which updates only the 2^{n-1} amplitude pairs differing in the target bit at $\mathcal{O}(2^n)$ cost per gate[cite: 1].

Because the trace is append-only and snapshots are immutable, both `inspector/` and `debugger/` are read-only leaves: they read `StepTrace` exclusively and never

mutate state. This decouples analysis UI from the engine and guarantees that inspector/debugger activity cannot corrupt an in-flight simulation, a prerequisite for the strict layering rule that leaf folders are imported only by `main.js` and injected via `GateRegistry`[cite: 1].

III. Architecture Decisions

- **Layering:** `core/` imports only internal `core/`; `circuit/` imports `core/` + internal `circuit/`; `sim/` imports `core/` + `circuit/`. Leaf folders (`gates/`, `debugger/`, `inspector/`) are imported exclusively by `main.js` and injected into `circuit/` via `GateRegistry`; zero imports exist between any two leaf folders[cite: 1].
- **State format:** split `Float64Array` buffers (`re[]`, `im[]`); gates flattened at definition time into contiguous 8-double tuples[cite: 1].
- **Zero mutations:** inspector and debugger perform read-only access via `currentState()`; no module-level mutable state anywhere[cite: 1].
- **CLI pipe contract:** headless runner reads canonical v1 circuit JSON from `stdin` and writes deterministic state-vector JSON to `stdout`; diagnostics go to `stderr`[cite: 1].
- **Testing:** every source module has a 1:1 mirrored `*.test.js` under `test/`; no test asserts hardcoded non-constant outputs[cite: 1].

IV. Validation

All verification was performed on bare Node.js via `node --test` from the repo root: the suite passes 100%, and `cat test/fixtures/bell_state.json | node cli/cli.js` produces correct Bell-state JSON on `stdout`[cite: 1]. QFT and Shor builders are differentially validated against `analytic.js` closed-form expectations within tolerance; the Serializer round-trip `decode(encode(c)).isEqualTo(c)` holds across generated circuits with subcircuits, controls, and anti-controls. A layering audit confirmed `gates/` is imported only by `main.js`, there are zero inter-leaf imports, and the `ai/` folder is absent[cite: 1].

V. Next Steps

1) Sprint 4: implement `src/ai/` (step 11 — local inference under the frozen `AI_INTERFACE.md` contract, strictly last) and the remaining `src/canvas/` work (`Interaction.js` plus dark-mode dockable panels, step 6) [**?**].
2) Extend the differential harness to parameterized gates (R_x, R_y, R_z) and multi-qubit controlled operations[cite: 1].
3) Explore Worker-thread parallelization of the kernel's block iteration over independent amplitude pairs[cite: 1].

Exit criterion: `node --test` passes 100%; every source module has a 1:1 test; CLI emits correct Bell-state output; no layering violations[cite: 1].

VI. Risks and Mitigations

1) **Memory ceiling:** ∼256 MB at 24 qubits; mitigated by `HARD_CLI_CAP=24` and advisory warnings at $15 \leq n \leq 20$[cite: 1].
2) **GC pressure:** mitigated by the zero-allocation kernel and Ephemeral Replay Mode detailed in Sections II–III[cite: 1].
3) **Bit-ordering inconsistencies:** mitigated by frozen conventions and automated bidirectional tests against the Kronecker reference[cite: 1].
4) **Numerical drift:** accumulated IEEE-754 rounding over deep circuits; mitigated by per-gate norm verification and selective renormalization at a 10^{-9} drift threshold[cite: 1].
5) **Layering drift:** new features may create forbidden inter-leaf imports; mitigated by a layering audit in the verification checklist[cite: 1].

References

[1] C. Gidney, "Quirk: A drag-and-drop quantum circuit simulator that runs in your browser," GitHub repository, Apache-2.0 license. [Online]. Available: https://github.com/strilanc/quirk

[2] A. Javadi-Abhari *et al.*, "Qiskit: An open-source framework for quantum computing," IBM, 2017. [Online]. Available: https://qiskit.org

[3] H. De Raedt and K. Michielsen, "Computational methods for simulating quantum computers," in *Quantum Computing and Quantum Bits in Mesoscopic Systems*. Hoboken, NJ, USA: Wiley, 2004.

[4] M. Smelyanskiy, N. P. D. Sawaya, and A. Aspuru-Guzik, "qHiPSTER: The Quantum High Performance Software Toolkit," 2016, *arXiv:1601.07195*.

[5] A. W. Cross *et al.*, "OpenQASM 3 specification," 2021. [Online]. Available: https://openqasm.com